

Tutorial paso a paso del algoritmo de intersección PRCG $\cap$ regular

Una guía visual con un ejemplo trabajado completo

Propósito de este documento. El Capítulo 2 de la tesis define un algoritmo que, dada una PRCG G y un autómata finito determinista A , construye una nueva PRCG G' tal que $L(G') = L(G) \cap L(A)$. Este documento presenta el algoritmo paso a paso sobre un ejemplo concreto, desarrollando todas las cláusulas y trazando una derivación completa al final.

1 El problema en una frase

Se parte de una PRCG G que genera un lenguaje $L(G)$, y un AFD A que reconoce un lenguaje regular $L(A)$. Se desea construir una PRCG G' tal que el lenguaje que genera G' sea exactamente $L(G) \cap L(A)$, esto es, las cadenas que pertenecen a ambos lenguajes simultáneamente.

Idea fundamental. En lugar de razonar sobre rangos $\langle i, j \rangle$ que apuntan a posiciones de una cadena fija w , la construcción razona sobre *pares de estados* $\langle p, p' \rangle$ del autómata A . Un par $\langle p, p' \rangle$ representa una subcadena que lleva A del estado p al estado p' .

2 Hipótesis sobre la PRCG de entrada

El algoritmo del presente tutorial trata las PRCG *top-down non-erasing*, en el sentido de Boullier [?]: una PRCG G es top-down non-erasing si, para toda cláusula c de G , cada variable que aparece en la cabeza de c aparece también en el cuerpo de c . La restricción es sintáctica y se verifica por inspección directa de las cláusulas.

La restricción no es una limitación esencial. Por la cadena de Properties 1, 2 y 3 de Boullier [?], toda PRCG admite una versión equivalente que es no-combinatorial y non-erasing, la cual es en particular top-down non-erasing. La transformación que produce la versión equivalente queda fuera del alcance del tutorial: la gramática que sirve de entrada al algoritmo se asume top-down non-erasing.

La clase top-down non-erasing es estrictamente más amplia que la clase sRCG tratada por Bertsch y Nederhof [?]: admite cláusulas no-lineales, como las del ejemplo motivador que se desarrolla en la sección siguiente, y cláusulas combinatorias, en las que un argumento del cuerpo contiene terminales mezclados con variables.

3 Ejemplo de partida

3.1 La gramática G

Sea $G = (N, T, V, P, S)$ con $N = \{S, \text{Par}\}$, $T = \{a\}$, $V = \{X\}$, axioma S , y las cláusulas siguientes:

$$\begin{aligned} c_1 : & S(X) \rightarrow \text{Par}(X) \text{Par}(X) \\ c_2 : & \text{Par}(aaX) \rightarrow \text{Par}(X) \\ c_3 : & \text{Par}(\varepsilon) \rightarrow \varepsilon \end{aligned}$$

Qué genera esta gramática. Cada cláusula tiene un rol específico:

- c_1 : el predicado S deriva una cadena X que pasa simultáneamente por dos invocaciones de Par . La variable X aparece tres veces en la cláusula: una en la cabeza y dos en el cuerpo. La semántica de PRCG exige que las tres ocurrencias representen la misma cadena.
- c_2 : Par aplicada a una cadena que empieza con aa se reduce a Par aplicada al resto.
- c_3 : Par acepta la cadena vacía.

Las cláusulas c_2 y c_3 definen Par como el conjunto de cadenas sobre $\{a\}$ con longitud par, esto es, $\{a^{2n} : n \geq 0\}$. La cláusula c_1 exige que la cadena entera sea aceptada por Par de manera simultánea por las dos invocaciones que deben coincidir. Por tanto $L(G) = \{a^{2n} : n \geq 0\}$.

No-linealidad top-down. La variable X aparece dos veces en el cuerpo, una vez en cada Par . Esto rompe la condición de *linealidad top-down* de Boullier [?], Definición 8: ninguna variable puede aparecer más de una vez en la unión de los argumentos del cuerpo. La cláusula c_1 es **no-lineal top-down**, y la situación en la que la condición **C2** de la construcción actúa de manera no trivial.

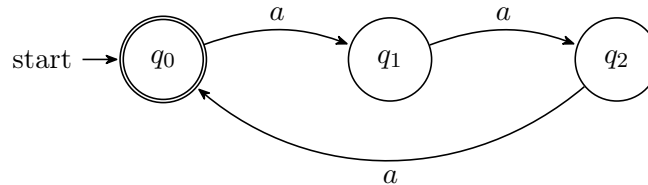
Top-down non-erasing. La gramática G_0 cumple la hipótesis del tutorial: en cada cláusula, toda variable de la cabeza aparece también en el cuerpo. En c_1 , la variable X aparece en la cabeza y en los dos predicados del cuerpo. En c_2 , la variable X aparece en la cabeza y en el único predicado del cuerpo. En c_3 , no hay variables. La hipótesis se cumple por tanto en las tres cláusulas.

3.2 El autómata A

Sea $A = (Q, T, \delta, q_0, F)$ con $Q = \{q_0, q_1, q_2\}$, $T = \{a\}$, estado inicial q_0 y estados finales $F = \{q_0\}$. Las transiciones son

$$\delta(q_0, a) = q_1, \quad \delta(q_1, a) = q_2, \quad \delta(q_2, a) = q_0.$$

El autómata se representa gráficamente como sigue.



El autómata A acepta una cadena si y solo si su longitud es múltiplo de 3:

$$L(A) = \{a^{3k} : k \geq 0\} = \{\varepsilon, aaa, aaaaaa, a^9, a^{12}, \dots\}.$$

3.3 La intersección por construir

Una cadena pertenece a $L(G) \cap L(A)$ si y solo si pertenece a ambos lenguajes:

$$L(G) \cap L(A) = \{a^{2n} : n \geq 0\} \cap \{a^{3k} : k \geq 0\} = \{a^{6m} : m \geq 0\}.$$

El conjunto resultante son las cadenas de a con longitud múltiplo de 6: $\{\varepsilon, aaaaaa, a^{12}, a^{18}, \dots\}$.

4 La idea de la construcción

Cada predicado B de G con aridad k se reemplaza por una colección de versiones indexadas $B[\vec{q}]$, donde $\vec{q}$ es un vector con k pares de estados, uno por cada argumento de B .

El predicado $B[\langle p, p' \rangle]$ aplicado al rango $\langle i, j \rangle$ deriva exactamente cuando:

- el predicado B original aplicado al mismo rango deriva en G , y
- el autómata A leyendo la subcadena $w[i + 1 \dots j]$ va de p a p' .

5 Construcción de G' paso a paso

Paso 1 (Crear los predicados indexados).

El conjunto N' de no terminales de G' se construye así:

- Un nuevo símbolo distinguido S' de aridad 1.
- Por cada predicado $B \in N$ de aridad k y cada vector $\vec{q} \in (Q \times Q)^k$, un nuevo símbolo $B[\vec{q}]$ de aridad k .

En el ejemplo, S y Par tienen aridad 1, así que cada uno produce $|Q|^2 = 9$ versiones indexadas. El total es $1 + 9 + 9 = 19$ símbolos no terminales.

Paso 2 (Cláusulas iniciales: conectar S' con S indexada).

Por cada estado final $q \in F$ se añade una cláusula

$$S'(X) \rightarrow S[\langle q_0, q \rangle](X).$$

Esta cláusula expresa que S' acepta X siempre que S leyendo X vaya del estado inicial q_0 a algún estado final q . Esa es la condición precisa para que X sea aceptada por el autómata A .

En el ejemplo, $F = \{q_0\}$, así que solo se añade una cláusula inicial:

Cláusula inicial: $S'(X) \rightarrow S[\langle q_0, q_0 \rangle](X)$

Paso 3 (Cláusulas heredadas de c_1 — donde **C2** actúa).

La cláusula original es $c_1 : S(X) \rightarrow \text{Par}(X) \text{Par}(X)$. La construcción genera todas las instancias indexadas que satisfagan las condiciones **C1** y **C2**.

- **C1 (coherencia interna):** en cada argumento de la cláusula, las transiciones del autómata sobre los terminales y los pares asignados a las variables deben encajar.
- **C2 (coherencia entre ocurrencias):** si una variable X aparece varias veces en la cláusula, todas sus ocurrencias deben llevar el mismo par de estados.

La variable X aparece tres veces en c_1 : una en la cabeza y dos en los predicados Par del cuerpo. Por **C2**, las tres ocurrencias deben tener el **mismo** par de estados. Sea $\Phi(X) = \langle p, p' \rangle$ ese par único.

Aplicando **C1** al argumento de la cabeza, que consta únicamente de la variable X , el par del argumento de la cabeza coincide directamente con $\langle p, p' \rangle$. Lo mismo vale para los argumentos de los dos Par en el cuerpo: ambos llevan el par $\langle p, p' \rangle$.

La cláusula instanciada es

$$S[\langle p, p' \rangle](X) \rightarrow \text{Par}[\langle p, p' \rangle](X) \text{Par}[\langle p, p' \rangle](X).$$

Como $\Phi(X)$ recorre los 9 pares posibles, se generan 9 cláusulas heredadas a partir de c_1 . La cláusula alcanzable desde la cláusula inicial $S'(X) \rightarrow S[\langle q_0, q_0 \rangle](X)$ es

Heredada alcanzable de c_1 :

$$S[\langle q_0, q_0 \rangle](X) \rightarrow \text{Par}[\langle q_0, q_0 \rangle](X) \text{Par}[\langle q_0, q_0 \rangle](X)$$

Consecuencia de omitir C2. Sin **C2**, las tres ocurrencias de X podrían recibir pares distintos. Por ejemplo, una cláusula heredada que viola **C2** sería

$$S[\langle q_0, q_0 \rangle](X) \rightarrow \text{Par}[\langle q_0, q_1 \rangle](X) \text{Par}[\langle q_1, q_0 \rangle](X),$$

en la que la misma X tendría que ser una cadena que vaya simultáneamente de q_0 a q_1 y de q_1 a q_0 , situación imposible para una sola subcadena. La verificación operacional reportada en el Capítulo 2 confirma que sin **C2**, la gramática producto sobre-genera con cadenas como aa y $aaaa$, que pertenecen a $L(G)$ pero no a $L(A)$.

Paso 4 (Cláusulas heredadas de c_2 — consumo de terminales).

La cláusula original es $c_2 : \text{Par}(aaX) \rightarrow \text{Par}(X)$. La variable X tiene dos ocurrencias, una en la cabeza dentro del argumento aaX y otra en el cuerpo. Por **C2**, ambas reciben el mismo par $\Phi(X) = \langle p, p' \rangle$.

C1 aplicada al argumento de la cabeza, $\alpha = aaX$, descompuesto como $\beta_1 = a$, $\beta_2 = a$, $\beta_3 = X$, exige una sucesión de estados $r_0 = p_0$, r_1 , r_2 , $r_3 = p'_0$ tal que

- $\beta_1 = a$ implica $\delta(r_0, a) = r_1$,
- $\beta_2 = a$ implica $\delta(r_1, a) = r_2$,
- $\beta_3 = X$ implica $\langle r_2, r_3 \rangle = \langle p, p' \rangle$, esto es $r_2 = p$ y $r_3 = p'$.

Combinando las tres condiciones, $\delta(\delta(p_0, a), a) = p$. El estado p_0 es el predecesor de p por dos transiciones de a . En el autómata A , dos transiciones de a corresponden a un avance de dos estados, que se condensa en la siguiente tabla.

p	$\text{pred}^2(p) = p_0$ tal que $\delta^2(p_0, aa) = p$
q_0	q_1
q_1	q_2
q_2	q_0

Para cada elección de $\Phi(X) = \langle p, p' \rangle$, con 9 opciones, la cláusula instanciada es

$$\text{Par}[\langle \text{pred}^2(p), p' \rangle](aaX) \rightarrow \text{Par}[\langle p, p' \rangle](X).$$

Las 9 cláusulas heredadas de c_2 se enumeran a continuación.

Heredadas de c_2 :

$$\begin{aligned}
& \text{Par}[\langle q_1, q_0 \rangle](aaX) \rightarrow \text{Par}[\langle q_0, q_0 \rangle](X) \\
& \text{Par}[\langle q_1, q_1 \rangle](aaX) \rightarrow \text{Par}[\langle q_0, q_1 \rangle](X) \\
& \text{Par}[\langle q_1, q_2 \rangle](aaX) \rightarrow \text{Par}[\langle q_0, q_2 \rangle](X) \\
& \text{Par}[\langle q_2, q_0 \rangle](aaX) \rightarrow \text{Par}[\langle q_1, q_0 \rangle](X) \\
& \text{Par}[\langle q_2, q_1 \rangle](aaX) \rightarrow \text{Par}[\langle q_1, q_1 \rangle](X) \\
& \text{Par}[\langle q_2, q_2 \rangle](aaX) \rightarrow \text{Par}[\langle q_1, q_2 \rangle](X) \\
& \text{Par}[\langle q_0, q_0 \rangle](aaX) \rightarrow \text{Par}[\langle q_2, q_0 \rangle](X) \\
& \text{Par}[\langle q_0, q_1 \rangle](aaX) \rightarrow \text{Par}[\langle q_2, q_1 \rangle](X) \\
& \text{Par}[\langle q_0, q_2 \rangle](aaX) \rightarrow \text{Par}[\langle q_2, q_2 \rangle](X)
\end{aligned}$$

Paso 5 (Cláusulas terminales heredadas de c_3).

La cláusula original es $c_3 : \text{Par}(\varepsilon) \rightarrow \varepsilon$. La condición **C1** aplicada al argumento ε obliga a que el par asignado tenga ambas componentes iguales, $p_0 = p'_0$, ya que la cadena vacía no produce transición. Hay 3 cláusulas heredadas, una por cada estado de Q .

Heredadas de c_3 :

$$\begin{aligned}
& \text{Par}[\langle q_0, q_0 \rangle](\varepsilon) \rightarrow \varepsilon \\
& \text{Par}[\langle q_1, q_1 \rangle](\varepsilon) \rightarrow \varepsilon \\
& \text{Par}[\langle q_2, q_2 \rangle](\varepsilon) \rightarrow \varepsilon
\end{aligned}$$

6 La gramática G' completa

Recopilando todos los pasos, G' contiene los siguientes elementos:

- **19 símbolos no terminales:** 1 axioma, 9 versiones de S y 9 versiones de Par .
- **22 cláusulas:** 1 inicial, 9 heredadas de c_1 , 9 heredadas de c_2 y 3 heredadas de c_3 .

7 Definición formal de las condiciones C1 y C2

Antes de presentar el algoritmo conviene fijar las dos condiciones que se han usado de manera informal en los pasos anteriores. La presentación que sigue formaliza ambas condiciones como predicados sobre asignaciones de pares de estados a las variables de una cláusula.

Notación previa. Sea c una cláusula de la forma

$$B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1(\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m(\alpha_{m,1}, \dots, \alpha_{m,k_m}),$$

donde cada $B_j \in N$ tiene aridad k_j , y cada argumento $\alpha_{j,i}$ es una sucesión de la forma $\beta_{j,i}^1 \beta_{j,i}^2 \cdots \beta_{j,i}^{l_{j,i}}$ con $\beta_{j,i}^u \in T \cup V$.

El conjunto de variables que aparecen en c se denota V_c . Es el subconjunto del conjunto V de variables de la gramática formado por las variables con alguna ocurrencia en c .

Una *asignación* es una función $\Phi : V_c \rightarrow Q \times Q$. La asignación asocia a cada variable un par de estados del autómata. El conjunto de todas las asignaciones posibles es $(Q \times Q)^{V_c}$, con cardinalidad $|Q|^{2|V_c|}$.

Representación operacional de Φ . Para que el algoritmo sea aplicable en la práctica, se fija a continuación cómo se representa Φ como objeto computable y cómo se computan las operaciones que el algoritmo necesita sobre Φ .

- Se fija una enumeración $(X_1, X_2, \dots, X_n)$ de V_c , donde $n = |V_c|$. La enumeración se construye listando los nombres de las variables en orden lexicográfico, o en el orden en que aparecen al recorrer la cláusula c de izquierda a derecha; cualquier orden fijo sirve, siempre que se mantenga el mismo en todas las consultas.
- Una asignación Φ se representa como una tupla de n pares de estados,

$$(\langle p_1, p'_1 \rangle, \langle p_2, p'_2 \rangle, \dots, \langle p_n, p'_n \rangle) \in (Q \times Q)^n.$$

La t -ésima componente de la tupla es el par asignado a la variable X_t .

- La consulta $\Phi(X)$, donde X es una variable de V_c , se computa así: se busca el índice t tal que $X = X_t$ en la enumeración fija, y se devuelve la t -ésima componente de la tupla. La operación corresponde a un acceso indexado en una tabla o un diccionario.
- La enumeración de todas las asignaciones posibles se hace recorriendo el producto cartesiano $(Q \times Q)^n$. Cada tupla del producto se interpreta como una asignación según la regla del punto anterior. Hay $|Q|^{2n}$ tuplas distintas, generables mediante n bucles anidados sobre $Q \times Q$ o mediante cualquier rutina estándar de generación de producto cartesiano.

Ejemplo. Si $V_c = \{X, Y\}$ y la enumeración fijada es (X, Y) , la tupla $(\langle q_0, q_1 \rangle, \langle q_2, q_0 \rangle)$ representa la asignación con $\Phi(X) = \langle q_0, q_1 \rangle$ y $\Phi(Y) = \langle q_2, q_0 \rangle$.

Condición C2 (formalmente). La condición **C2** está incorporada en la propia definición de Φ como función. Dado que Φ asocia exactamente un par a cada variable, todas las ocurrencias de la misma variable en c reciben automáticamente el mismo par. La formulación “ Φ es una función de V_c en $Q \times Q$ ” equivale a imponer **C2** desde el inicio del razonamiento. La incorporación se hace explícita en el algoritmo más adelante.

Condición C1 (formalmente). Dado un argumento $\alpha_{j,i} = \beta_{j,i}^1 \beta_{j,i}^2 \dots \beta_{j,i}^{l_{j,i}}$ y una asignación Φ , el argumento $\alpha_{j,i}$ se dice *C1-compatible bajo Φ con par $\langle p_0, p_l \rangle$* si existe una sucesión de estados $r_0, r_1, \dots, r_{l_{j,i}}$ tal que $r_0 = p_0$, $r_{l_{j,i}} = p_l$, y para cada $u \in \{1, \dots, l_{j,i}\}$:

- si $\beta_{j,i}^u = a \in T$, entonces $\delta(r_{u-1}, a) = r_u$;
- si $\beta_{j,i}^u = X \in V$, entonces $\langle r_{u-1}, r_u \rangle = \Phi(X)$.

El par $\langle p_0, p_l \rangle$ se llama *par inducido por Φ sobre el argumento $\alpha_{j,i}$* . La cláusula c se dice *C1-compatible bajo Φ* si cada uno de sus argumentos $\alpha_{j,i}$ admite al menos un par inducido. La asignación Φ produce una cláusula heredada por cada elección de pares inducidos para los argumentos.

Intuición. La condición **C1** expresa que las transiciones del autómata sobre los terminales del argumento y los pares de las variables encajan en una sucesión consistente de estados. La condición **C2** expresa que cada variable lleva el mismo par en todas sus ocurrencias. La construcción genera una cláusula heredada de c exactamente cuando ambas condiciones se cumplen para una asignación Φ y una elección de pares inducidos.

8 Pseudocódigo del algoritmo

Esta sección presenta el algoritmo completo. Está organizado en cuatro subrutinas: la rutina principal CONSTRUIRGPIMA, que recibe G y A y devuelve G'' ; la subrutina CLAUSULASHEREDADAS, que genera las cláusulas heredadas de una cláusula dada; la subrutina ENUMERAR, que recorre el producto

cartesiano de los conjuntos de pares inducidos; y la subrutina PARESINDUCIDOS, que calcula los pares de estados consistentes con un argumento bajo una asignación.

Aviso sobre notación. En el pseudocódigo se usa la letra B para los predicados de N y se reserva la letra A para el autómata. La cláusula c se escribe usando $B_0, B_1, \dots, B_m$ como nombres genéricos de los predicados que aparecen en ella.

Convenciones del pseudocódigo. Las subrutinas son funciones puras: ninguna modifica sus argumentos ni produce efectos secundarios. Los bucles de la forma “**for** cada $x \in S$ **do**” recorren el conjunto S en un orden lexicográfico fijo sobre una representación canónica de sus elementos; la corrección de los algoritmos no depende del orden particular escogido, pero la elección de un orden fijo garantiza que la salida es determinista. El autómata A se asume con función de transición δ posiblemente parcial; cuando $\delta(q, a)$ no está definido para un par (q, a) , las cláusulas heredadas asociadas no se generan.

Algoritmo 1 CONSTRUIRPRIMA(G, A)

Entrada: PRCG $G = (N, T, V, P, S)$ y AFD $A = (Q, T, \delta, q_0, F)$ sobre el mismo alfabeto T .

Salida: PRCG $G' = (N', T, V, P', S')$ con $L(G') = L(G) \cap L(A)$.

```

1:                                     ▷ Paso 1: crear los predicados indexados
2:  $N' \leftarrow \{S'\}$ 
3: for cada  $B \in N$  do
4:   sea  $k$  la aridad de  $B$ 
5:   for cada vector  $\vec{q} = (\langle p_1, p'_1 \rangle, \dots, \langle p_k, p'_k \rangle) \in (Q \times Q)^k$  do
6:     añadir  $B[\vec{q}]$  a  $N'$ , con aridad  $k$ 
7:   end for
8: end for
9:                                     ▷ Paso 2: cláusulas iniciales
10:  $P' \leftarrow \emptyset$ 
11: for cada  $q_f \in F$  do
12:   añadir la cláusula  $S'(X) \rightarrow S[\langle q_0, q_f \rangle](X)$  a  $P'$ 
13: end for
14:                                     ▷ Paso 3: cláusulas heredadas
15: for cada cláusula  $c \in P$  do
16:    $H_c \leftarrow \text{CLAUSULASHEREDADAS}(c, A)$ 
17:    $P' \leftarrow P' \cup H_c$ 
18: end for
19: return  $G' = (N', T, V, P', S')$ 

```

La subrutina CLAUSULASHEREDADAS genera todas las cláusulas heredadas de una cláusula original c . La estructura es: enumerar las asignaciones Φ , y por cada Φ calcular los pares inducidos sobre cada argumento.

Algoritmo 2 CLAUSULASHEREDADAS(c, A)

Entrada: Cláusula $c = B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1(\alpha_{1,1}, \dots) \cdots B_m(\alpha_{m,1}, \dots)$, autómata A .

Salida: Conjunto H_c de cláusulas heredadas de c .

```
1:  $H_c \leftarrow \emptyset$ 
2:  $V_c \leftarrow \{X \in V : X \text{ aparece en alguna posición de } c\}$  ▷ las variables presentes en  $c$ 
3:  $(X_1, X_2, \dots, X_n) \leftarrow$  una enumeración fija de  $V_c$  con  $n = |V_c|$ 
4: for cada tupla  $\tau = (\langle p_1, p'_1 \rangle, \dots, \langle p_n, p'_n \rangle) \in (Q \times Q)^n$  do ▷ producto cartesiano
5:    $\Phi \leftarrow$  la asignación con  $\Phi(X_t) = \langle p_t, p'_t \rangle$  para cada  $t \in \{1, \dots, n\}$  ▷ C2 incorporada por construcción
6:    $paresArgs \leftarrow$  tabla vacía indexada por pares  $(j, i)$ 
7:    $ok \leftarrow$  verdadero
8:   for  $j = 0, 1, \dots, m$  do ▷ recorre los predicados  $B_0, B_1, \dots, B_m$  de la cláusula
9:     for  $i = 1, 2, \dots, k_j$  do ▷ recorre los argumentos del predicado  $B_j$ 
10:      if  $\neg ok$  then
11:        continue ▷  $\Phi$  ya está descartada; el bucle externo continúa
12:      end if
13:       $candidatos \leftarrow \text{PARESINDUCIDOS}(\alpha_{j,i}, \Phi, A)$ 
14:      if  $candidatos = \emptyset$  then ▷ C1 falla sobre este argumento
15:         $ok \leftarrow$  falso
16:      else
17:         $paresArgs[(j, i)] \leftarrow candidatos$ 
18:      end if
19:    end for
20:  end for
21:  if  $ok$  then
22:     $L \leftarrow$  la lista de pares de índices  $(j, i)$  con  $j \in \{0, \dots, m\}$  e  $i \in \{1, \dots, k_j\}$ , ordenada lexicográficamente
23:     $combinaciones \leftarrow \text{ENUMERAR}(paresArgs, L)$  ▷ enumera el producto cartesiano
24:    for cada  $parElegido \in combinaciones$  do ▷  $parElegido[(j, i)]$  es el par escogido para  $(j, i)$ 
25:      for  $j = 0, 1, \dots, m$  do
26:         $\vec{q}_j \leftarrow (parElegido[(j, 1)], parElegido[(j, 2)], \dots, parElegido[(j, k_j)])$ 
27:      end for
28:       $h \leftarrow B_0[\vec{q}_0](\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1[\vec{q}_1](\alpha_{1,1}, \dots) \cdots B_m[\vec{q}_m](\alpha_{m,1}, \dots)$ 
29:       $H_c \leftarrow H_c \cup \{h\}$ 
30:    end for
31:  end if
32: end for
33: return  $H_c$ 
```

La enumeración del producto cartesiano $(Q \times Q)^n$ en la línea 4 produce $|Q|^{2n}$ tuplas. La enumeración fija $(X_1, \dots, X_n)$ establece la correspondencia entre la posición de cada par en la tupla y la variable a la que se asigna. La cantidad de iteraciones del bucle externo está acotada por $|Q|^{2n}$, cota que demuestra la terminación del bucle. La invocación a `ENUMERAR` en la línea 23 enumera un producto cartesiano con una cantidad de elementos acotada por $|Q|^{2K_c}$, donde $K_c = \sum_j k_j$ es la suma de las aridades de los predicados de c .

La subrutina `ENUMERAR` recorre el producto cartesiano de los conjuntos $paresArgs[(j, i)]$ en el orden establecido por la lista L . Cada elemento de la enumeración es una asignación que, para cada

$(j, i) \in L$, fija un par de $paresArgs[(j, i)]$.

Algoritmo 3 ENUMERAR($paresArgs, L$)

Entrada: Tabla $paresArgs$ indexada por pares (j, i) , lista ordenada $L = ((j_1, i_1), \dots, (j_r, i_r))$ de los pares de índices.

Salida: Conjunto de todas las funciones $parElegido : L \rightarrow Q \times Q$ tales que $parElegido[(j_s, i_s)] \in paresArgs[(j_s, i_s)]$ para cada $s \in \{1, \dots, r\}$.

```

1:  $resultado \leftarrow \emptyset$ 
2: if  $L$  es la lista vacía then
3:    $resultado \leftarrow \{vacía\}$  ▷ una única función vacía
4:   return  $resultado$ 
5: end if
6: sea  $(j_1, i_1)$  el primer elemento de  $L$  y  $L' = ((j_2, i_2), \dots, (j_r, i_r))$  el resto
7:  $subresultado \leftarrow$  ENUMERAR( $paresArgs, L'$ )
8: for cada par  $\langle p, p' \rangle \in paresArgs[(j_1, i_1)]$  do
9:   for cada  $f \in subresultado$  do
10:     $g \leftarrow f$  extendida con  $g[(j_1, i_1)] = \langle p, p' \rangle$ 
11:     $resultado \leftarrow resultado \cup \{g\}$ 
12:   end for
13: end for
14: return  $resultado$ 

```

La recursión de ENUMERAR termina después de exactamente $|L|$ llamadas recursivas, cuando la lista L se reduce a la vacía. La cantidad total de funciones generadas es $\prod_{(j,i) \in L} |paresArgs[(j, i)]|$, acotada por $|Q|^{2K_c}$.

La subrutina PARESINDUCIDOS se define a continuación. Recibe un argumento α , una asignación Φ y el autómata A , y devuelve el conjunto de pares $\langle p_0, p_l \rangle$ que son C1-compatibles con α bajo Φ , según la definición de la sección anterior.

Algoritmo 4 PARESINDUCIDOS(α, Φ, A)

Entrada: Argumento $\alpha = \beta^1 \beta^2 \dots \beta^l$, asignación Φ , autómata $A = (Q, T, \delta, q_0, F)$.

Salida: Conjunto de pares $\langle p_0, p_l \rangle$ que son C1-compatibles con α bajo Φ .

```
1: candidatos  $\leftarrow \emptyset$ 
2: if  $\alpha = \varepsilon$  then                                 $\triangleright$  argumento vacío: el par debe tener ambas componentes iguales
3:   for cada  $q \in Q$  do
4:     añadir  $\langle q, q \rangle$  a candidatos
5:   end for
6:   return candidatos
7: end if
8: for cada  $p_0 \in Q$  do                                 $\triangleright$  se enumera el estado inicial del argumento
9:    $r \leftarrow p_0$ 
10:  compatible  $\leftarrow$  verdadero
11:  for  $u = 1, 2, \dots, l$  do
12:    if  $\beta^u = a$  es un terminal de  $T$  then
13:      if  $\delta(r, a)$  no está definido then
14:        compatible  $\leftarrow$  falso; break
15:      end if
16:       $r \leftarrow \delta(r, a)$ 
17:    else if  $\beta^u = X$  es una variable de  $V$  then
18:      sea  $\langle p, p' \rangle = \Phi(X)$ 
19:      if  $r \neq p$  then                                 $\triangleright$  el par asignado a  $X$  no encaja con el estado actual
20:        compatible  $\leftarrow$  falso; break
21:      end if
22:       $r \leftarrow p'$ 
23:    end if
24:  end for
25:  if compatible then
26:    añadir  $\langle p_0, r \rangle$  a candidatos
27:  end if
28: end for
29: return candidatos
```

La subrutina PARESINDUCIDOS simula el recorrido del autómata sobre el argumento, símbolo por símbolo, comenzando desde cada estado inicial posible. Los terminales se procesan aplicando δ . Las variables se procesan obligando a que el estado actual coincida con la primera componente del par asignado y avanzando al estado de la segunda componente. Si todo el argumento se procesa sin fallo, el par $\langle p_0, r \rangle$ con el estado inicial p_0 y el estado final r es **C1-compatible**. La enumeración recorre los $|Q|$ valores posibles de p_0 .

Notas sobre el algoritmo.

- Cada tupla del producto cartesiano $(Q \times Q)^n$ corresponde a exactamente una asignación Φ . La correspondencia se establece según la enumeración fija $(X_1, \dots, X_n)$: la t -ésima componente de la tupla define $\Phi(X_t)$. La función Φ no se construye explícitamente como un diccionario, sino que la propia tupla con la enumeración fija ya proporciona la consulta.
- La condición **C2** se obtiene de manera automática del diseño de Φ como función. No hay un paso explícito que verifique que dos ocurrencias de la misma variable tienen el mismo par, porque

tal requisito está garantizado por la estructura misma de Φ . Una implementación que omitiera **C2** tendría que tratar cada ocurrencia de cada variable como una variable independiente, que es justamente lo que hace la versión de control del experimento operacional reportado en el Capítulo 2.

- La subrutina PARESINDUCIDOS puede devolver más de un par cuando el argumento comienza con una variable, porque en ese caso el estado inicial p_0 no queda fijado por los terminales y se enumera. Cuando el argumento comienza con un terminal, los terminales fijan el estado inicial y queda un único par candidato.
- El número total de cláusulas heredadas a partir de una cláusula c está acotado por $|Q|^{2|V_c|} \cdot |Q|^{2K_c}$, donde K_c es la suma de las aridades de los predicados de c . El primer factor proviene de la enumeración de asignaciones, y el segundo de la elección de pares inducidos sobre los argumentos.

9 Traza de una derivación: $aaaaaa \in L(G) \cap L(A)$

A continuación se verifica que G' deriva la cadena $aaaaaa$ de longitud 6.

9.1 Paso preliminar: el recorrido de A sobre $aaaaaa$

El recorrido del autómata A sobre la cadena $w = aaaaaa$ fija los pares de estados que cada rango de w tendrá:

posición k	0	1	2	3	4	5	6
estado s_k	q_0	q_1	q_2	q_0	q_1	q_2	q_0

El par de cualquier rango $\langle i, j \rangle$ es $\langle s_i, s_j \rangle$.

9.2 Pares de estados de la derivación

Los pares de estados que aparecen en la derivación se calculan aplicando la regla anterior a los rangos relevantes.

rango $\langle i, j \rangle$	subcadena $w[i + 1 \dots j]$	par $\langle s_i, s_j \rangle$
$\langle 0, 6 \rangle$	$aaaaaa$	$\langle q_0, q_0 \rangle$
$\langle 2, 6 \rangle$	$aaaa$	$\langle q_2, q_0 \rangle$
$\langle 4, 6 \rangle$	aa	$\langle q_1, q_0 \rangle$
$\langle 6, 6 \rangle$	ε	$\langle q_0, q_0 \rangle$

9.3 La derivación en G' de $aaaaaa$

$$\begin{aligned}
S'(\langle 0, 6 \rangle) &\Rightarrow S[\langle q_0, q_0 \rangle](\langle 0, 6 \rangle) && [\text{cláusula inicial}] \\
&\Rightarrow \text{Par}[\langle q_0, q_0 \rangle](\langle 0, 6 \rangle) \text{Par}[\langle q_0, q_0 \rangle](\langle 0, 6 \rangle) && [c_1, \Phi(X) = \langle q_0, q_0 \rangle]
\end{aligned}$$

Cada uno de los dos $\text{Par}[\langle q_0, q_0 \rangle](\langle 0, 6 \rangle)$ se deriva de la misma manera.

$$\begin{aligned}
\text{Par}[\langle q_0, q_0 \rangle](\langle 0, 6 \rangle) &\Rightarrow \text{Par}[\langle q_2, q_0 \rangle](\langle 2, 6 \rangle) && [c_2 : \text{Par}[\langle q_0, q_0 \rangle](aaX) \rightarrow \text{Par}[\langle q_2, q_0 \rangle](X)] \\
&\Rightarrow \text{Par}[\langle q_1, q_0 \rangle](\langle 4, 6 \rangle) && [c_2] \\
&\Rightarrow \text{Par}[\langle q_0, q_0 \rangle](\langle 6, 6 \rangle) && [c_2] \\
&\Rightarrow \varepsilon && [c_3 : \text{Par}[\langle q_0, q_0 \rangle](\varepsilon) \rightarrow \varepsilon]
\end{aligned}$$

Cada paso usa una cláusula heredada de G' . Los pares de estados codifican el recorrido del autómata dentro de los predicados indexados, sin necesidad de mantener un trazado del autómata por separado.

9.4 Por qué G' rechaza $aaaa$

La cadena $aaaa$ pertenece a $L(G)$ porque tiene longitud par, pero no pertenece a $L(A)$ porque su longitud no es múltiplo de 3.

Recorrido de A sobre $w = aaaa$:

posición k	0	1	2	3	4
estado s_k	q_0	q_1	q_2	q_0	q_1

El par del rango $\langle 0, 4 \rangle$ es $\langle q_0, q_1 \rangle$, y q_1 **no es un estado final**. La única cláusula inicial es $S'(X) \rightarrow S[\langle q_0, q_0 \rangle](X)$, así que la derivación de $S'(\langle 0, 4 \rangle)$ tendría que pasar por $S[\langle q_0, q_0 \rangle]$ aplicado al rango $\langle 0, 4 \rangle$. Sin embargo, $S[\langle q_0, q_0 \rangle](\langle 0, 4 \rangle)$ deriva si y solo si $\delta^*(q_0, aaaa) = q_0$, que es falso. Por tanto $aaaa \notin L(G')$.

10 Resumen de los puntos clave

1. Cada predicado B de G se reemplaza por todas sus versiones indexadas $B[\vec{q}]$, una por cada vector de pares de estados con la aridad apropiada.
2. Las cláusulas iniciales conectan S' con $S[\langle q_0, q_f \rangle]$ para cada estado final $q_f \in F$.
3. Las cláusulas heredadas se generan recorriendo todas las asignaciones Φ posibles a las variables. La asignación Φ es una función, lo que obliga a asignar un único par por variable. **C2 se cumple por construcción.**
4. Por cada Φ , se calcula el par de cada argumento simulando el recorrido del autómata sobre los terminales del argumento e insertando los pares de las variables en la posición correspondiente. **C1 se verifica en ese cálculo.**
5. Si todos los argumentos producen pares válidos, se añade la cláusula heredada con los predicados indexados.
6. La condición **C2** es necesaria para que G' no sobre-genere. La verificación operacional del Capítulo 2 muestra que omitir **C2**, asignando pares por ocurrencia en lugar de por variable, produce sobre-generación incluso en gramáticas lineales en el sentido de Boullier [?].